scxmlcc

1 state

```
class state
{
    public:
    virtual ~state(){}
    virtual state* event_D(sc_vending_machine&) { return 0; }
    virtual state* event_N(sc_vending_machine&) { return 0; }
    virtual state* event_cancel(sc_vending_machine&) { return 0; }
    virtual state* event_coke(sc_vending_machine&) { return 0; }
    virtual state* event_diet(sc_vending_machine&) { return 0; }
    virtual state* event_done(sc_vending_machine&) { return 0; }
    virtual state* event_done_state(sc_vending_machine& sc) { return event_done(sc); }
    virtual state* event_done_state_active(sc_vending_machine& sc) { return event_done_state(sc); }
    virtual state* event_done_state_coin_return(sc_vending_machine& sc) { return event_done_state(sc); }
    virtual state* event_test(sc_vending_machine&) { return 0; }
    virtual state* event_zero(sc_vending_machine&) { return 0; }
    virtual state* unconditional(sc_vending_machine&) { return 0; }
    virtual state* initial(sc_vending_machine&) { return 0; }

    template<class T> void enter(data_model&, ...) {}
    template<class T> void exit(data_model&, ...) {}
    virtual void exit_to_src(data_model&, const void*) {}
    virtual bool in(const void*) { return false; }
};

typedef state* (state::*event)(sc_vending_machine&);
```

'state' is the base class for all states in the application. It provides a default implementation for each of the event handlers, including the 'initial' handler (for initial transition).

It also implements the ultimate (placeholder) methods for 'enter', 'exit', 'exit_to_src' and 'in' methods. They are called up the chain from their overriding methods in 'composite'. They simply provide a default return (do nothing) to break the chain. Note *exit_to_src()* and *in()* must be virtual to start the call chain from the most derived leaf state (see **transition** section). On the other hand, *enter()* and *exit()* must be non-virtual to start the call chain from the provided type which is either the source state (for exit) or destination state (for enter) of a transition (that is not necessarily a leaf state).

'event' is a pointer to member function of state.

2 state_actions

```
template<class C> class state_actions
{
    protected:
    void enter(data_model&) {} // default enter action
    void exit(data_model&) {} // default exit action
};
```

It allows an application to implement custom entry and exit actions via template specialization of the default methods.

```
template<> void sc::state_actions<sc::state_collect_coins>::enter(sc::data_model &m)
{
    // ready for coin insert
```

```
    m.user->sig_insert_coins(m.user->credit);
}
```

3 composite

```
    template<class C, class P> class composite : public P, public state_actions<C>
    {
        virtual state* initial(sc_vending_machine&) { return 0; }
        public:
        typedef P parent_t;
        // LCA calculation
        template<class T> void enter(data_model&, composite*) {}
        template<class T> void enter(data_model &m, ...) {
            P::template enter<T>(m, static_cast<T*>(nullptr)); state_actions<C>::enter(m);
        }
        template<class T> void exit(data_model&, composite*) {}
        template<class T> void exit(data_model &m, ...) {
            state_actions<C>::exit(m); P::template exit<T>(m, static_cast<T*>(nullptr));
        }
        virtual void exit_to_src(data_model &m, const void *sti) {
            if (C::id() == sti) return; state_actions<C>::exit(m); P::exit_to_src(m, sti);
        }
        static const void* id() { static const struct{} _id; return &_id; }
        bool in(const void *si) { return (si == id() || P::in(si)); }
    };
```

It is responsible for calling the chain of exit and entry actions for a transition. A transition is made up of three stages:

1. Exit actions from the current/leaf state up to *but not including* the transition source.
2. Exit actions from the transition source up to but not including the *least/lowest common ancestor* (LCA*), followed by entry actions from there into the transition target.

   *In general, we use the stricter *proper* definition of LCA, i.e. it is a proper superstate of the states under consideration. According to SCXML, there is an exception for transition type = internal and the source state a proper superstate of the destination state, in which case the LCA is the same as the source state which is NOT exited and re-entered. Note it does NOT apply to the case when the source state is a proper substate of the destination state, in which case the LCA is the immediate superstate of the destination state. In subsequent discussion, this exception case is denoted by EITT (Exception for Internal Transition Type)
3. Entry actions from *but not including* the transition target into the final state.

(1) and (2) are handled here, while (3) is handled in the dispatch_int() method (see later).

Resolving LCA at compile-time using overloaded method matching with automatic type conversion based on class hierarchy.

3.1 Exit from Current to Source

This method:

```
        virtual void exit_to_src(data_model &m, const void *sti) {
            if (C::id() == sti) return; state_actions<C>::exit(m); P::exit_to_src(m, sti);
        }
```

exits from the current (leaf) to (but not including) the source state. It is first called on the current leaf state (C) with *sti* set to the transition source (S). It exits the current state and iteratively calls the *exit_to_src()* method of its parent P until P == S (*sti*) in which case it terminates the call chain. Note S is not exited as it is done in the next stage.

It is guaranteed that the iterative calls to *exit_to_src()* will terminate, i.e. P in each iteration will eventually be equal to S (*sti*). Given the way a virtual function is selected, the current leaf state (C) must be a subclass of S for S's virtual event handler calling (or firing) the transition to be invoked. As a result, following C's parent iteratively will eventually reach S.

3.2 Exit from Source to LCA

For exit actions from source state up to (but not including) LCA, it relies on these two methods:

```
template<class T> void exit(data_model&, composite*) {}
template<class T> void exit(data_model &m, ...) {
    state_actions<C>::exit(m); P::template exit<T>(m, static_cast<T*>(nullptr));
}
```

P is the (type/class of the) parent state and C is the child state that we are now considering. T is the parent of the destination state (D) of the transition that is passed along the call chain. The purpose of T is to construct a placeholder object using *static_cast<T*>(nullptr)* to invoke one of the overloaded exit() method of the parent state P, taking advantage of the automatic type matching of the compiler.

If T can be automatically converted to P (including T==P) which is denoted as ***composite** ** in one of the function prototypes of P::template exit<T>, the following overloaded version will be selected and thus terminates the call chain.

```
template<class T> void exit(data_model&, composite*) {}
```

Since the state hierarchy is mirrored by the class hierarchy, if T (parent of D) can be converted to P (i.e. T <= P), D must be a substate of P (since D < T <= P => D < P). On the other hand, the source state (S) must be a substate of P since it starts calling *S::exit()* and iteratively calls its parent's version of *exit()*, i.e. S < P. In other words, S < P and D < P and P must be a common ancestor of S and D. By the fact that it starts the search with P being the *immediate* parent of S and that T is the *immediate* parent of D, the first match must be the least ancestor.

Note - In the call chain of *exit()*, the exit action of S is always called (as in *state_actions<C>::exit(m)*), while that of the LCA is never called.

3.3 Entry from LCA to Destination

For entry actions from (but not including) the LCA down to destination state, it relies on these two methods:

```
template<class T> void enter(data_model&, composite*) {}
template<class T> void enter(data_model &m, ...) {
    P::template enter<T>(m, static_cast<T*>(nullptr)); state_actions<C>::enter(m);
}
```

The analysis is similar to the case for the exit actions. It starts calling D::enter() with T set to the parent of S (considering the 'external' transition type). It iteratively calls one of the enter() methods of a transitive parent of D (P), i.e. D < P. It matches this overloaded version:

```
    template<class T> void enter(data_model&, composite*) {}
```

when T can be converted to P (of type *composite* *), i.e. when T <= P, which terminates the call chain. We have S < T <= P, and hence S < P. Together with D < P, P is the common ancestor of S and D. By the fact that it starts the search with P being the *immediate* parent of D and that T is the *immediate* parent of S, the first match must be the least ancestor.

4 transition_actions

```
    template<event E, class S, class D0 = no_state> class transition_actions
    {
        protected:
        void enter(data_model&) {} // default enter action
        bool condition(data_model&) { return true; } // default condition action
    };
```

It allows an application to implement custom transition actions and guards by template specialization of default class methods. The method name 'enter' is a misnomer, as it does not make sense to enter a transition. It should be interpreted as 'action'.

```
// Targeted transition action.
template<> void sc::transition_actions<&sc::state::event_D, sc::state_collect_coins,
    sc::state_collect_coins>::enter(sc::data_model &m)
{
    // dime inserted
    m.user->credit += dime;
}

// Targeted transition guard condition.
template<> bool sc::transition_actions<&sc::state::event_zero, sc::state_collect_coins,
    sc::state_dispense_zero>::condition(sc::data_model &m)
{
    // enough credit for zero?
    return m.user->credit >= price_zero;
}

// Targetless transition action.
template<> void sc::transition_actions<&sc::state::event_timer, sc::state_on>::enter(sc::data_model &m)
{
    ++m.user->timer;
}
```

5 transition

```
    // external/internal transition
    template<event E, class S, class D = no_state, transition_type T = external> class transition : public
        transition_actions<E, S, D>
    {
        template<transition_type I> struct id { };
        void state_enter(D* d, data_model &m, id<internal>, S*) {
            d->template enter<composite<S, typename S::parent_t> >(m);
        } // internal transition, where dst is descendant of src
        void state_enter(D* d, data_model &m, ...) {
            d->template enter<typename S::parent_t>(m);
        } // external transition, or dst is not descendant of src
        void state_exit(S*, data_model &, id<internal>, S*) {} // internal transition, where dst is descendant of src
        void state_exit(S* s, data_model &m, ...) {
```

```
            s->template exit<typename D::parent_t>(m);
        } // external transition, or dst is not descendant of src
        public:
        state* operator ()(S *s, sc_vending_machine &sc)
        {
            if(!transition_actions<E, S, D>::condition(sc.model)) return 0;
            D *d = sc.get_state<D>();
            s->exit_to_src(sc.model, S::id());
            state_exit(s, sc.model, id<T>(), static_cast<typename D::parent_t*>(nullptr));
            transition_actions<E, S, D>::enter(sc.model);
            state_enter(d, sc.model, id<T>(), static_cast<typename D::parent_t*>(nullptr));
            return d;
        }
    };
```

For each transition line, a transition object is created from the class (implicitly) instantiated from the transition class template with the event (E), source (S), destination (D) and type (T) specified. The created object is immediately 'called' to invoke its call operator which implements the transition rules in statechart. This happens in the overriding event handler of the triggering event in the source (S) state.

```
        state* operator ()(S *s, sc_vending_machine &sc)
        {
            if(!transition_actions<E, S, D>::condition(sc.model)) return 0;
            D *d = sc.get_state<D>();
            s->exit_to_src(sc.model, S::id());
            state_exit(s, sc.model, id<T>(), static_cast<typename D::parent_t*>(nullptr));
            transition_actions<E, S, D>::enter(sc.model);
            state_enter(d, sc.model, id<T>(), static_cast<typename D::parent_t*>(nullptr));
            return d;
        }
```

5.1 Exit to Source

It first calls *exit_to_src()* to exit from the current leaf state (pointed to by *s*) to (but not including) the transition source (S). It is important to note that an event handler and hence the transition *call operator* is called on the current state object (*model.cur_state*):

```
    private: bool dispatch_event(event e)
    {
        state *next_state;
        if ((next_state = (model.cur_state->*e)(*this))) model.cur_state = next_state;
        return !!next_state;
    }
```

As a result, though the parameter *s* is of type *S* *, it is indeed pointing to the current leaf state object, which is automatically upcasted to its (transitive) parent state S. See **Event Resolution** below. Since *exit_to_src()* is a virtual function, the version invoked is the overriding one in the composite<> class for/above the current leaf state pointed to by *s*:

```
    s->exit_to_src(sc.model, S::id());
```

This explains why *exit_to_src()* can call the exit actions from the leaf state up to (but not including) S.

5.2 Exit to LCA

The exit chain is initiated by this call in the call operator:

```
    state_exit(s, sc.model, id<T>(), static_cast<typename D::parent_t*>(nullptr));
```

There are two overloaded versions of state_exit():

```
void state_exit(S*, data_model &, id<internal>, S*) {} // internal transition, where dst is descendant of src
void state_exit(S* s, data_model &m, ...) {
    s->template exit<typename D::parent_t>(m);
} // external transition, or dst is not descendant of src
```

The first version is selected when (i) the parent of the destination state (D) can be converted to S (i.e. D < S, or D is a proper substate of S), and (ii) the transition type is *internal*. According to SCXML rules, S is not exited and re-entered in this case.

Otherwise the second version is selected in which case the exit<parent of D>() method of the source state (S) is called (precisely of the composite<> class for/above S). Note that the composite<>::exit<> method is non-virtual and therefore the exact version for S is called rather than that of the leaf state. This is important since the stage above has already taken care of exiting from the leaf state up to (but not including) S. The parent of the destination D is used to instantiate the exit<> method, which is used to find the LCA (see **composite** section).

5.3 Entry from LCA

The entry chain is initiated by this call in the call operator:

```
state_enter(d, sc.model, id<T>(), static_cast<typename D::parent_t*>(nullptr));
```

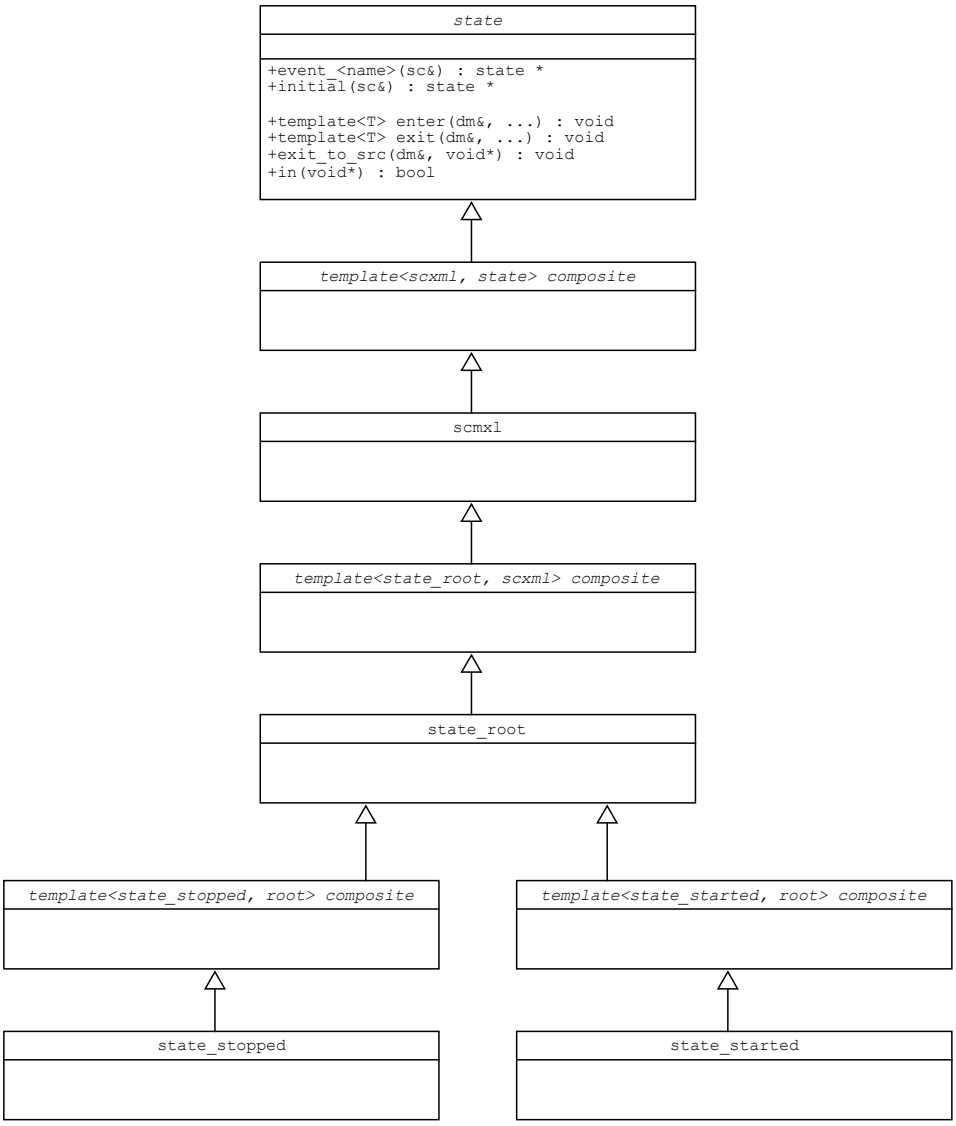
There are two overloaded versions of state_enter():

```
void state_enter(D* d, data_model &m, id<internal>, S*) {
    d->template enter<composite<S, typename S::parent_t> >(m);
} // internal transition, where dst is descendant of src
void state_enter(D* d, data_model &m, ...) {
    d->template enter<typename S::parent_t>(m);
} // external transition, or dst is not descendant of src
```

The first version is selected when (i) the parent of the destination state (D) can be converted to S (i.e. D < S, or D is a proper substate of S), and (ii) the transition type is *internal*. According to SCXML rules, S is not exited and *re-entered* in this case. To avoid re-entering S, it passes S (precisely the composite<> state above/for it, but equivalent) rather than its parent as the template parameter for enter<>. See **composite** section for details but essentially it matches S as the (improper) LCA and thus avoids S from being exited.

Otherwise the second version is selected in which case the enter<parent of S>() method of the destination state (D) is called (precisely of the composite<> class for/above D) . Note that the composite<>::enter<> method is non-virtual and therefore the exact version for D is called rather than that of the new leaf state. This is important since the initial transitions to the new leaf state will be taken care of in *dispatch_int()* outside the transition *call operator* and event handler. The parent of the source S is used to instantiate the enter<> method, which is used to find the LCA (see **composite** section).

6 state class hierarchy



The state hierarchy is mirrored by a similarly structured class hierarchy with *state* serving as the abstract base class. Each concrete state is added to the hierarchy using the CRTP (Curiously Recurring Template Pattern) with *composite*<> serving as the intermediate base class. The first concrete state is *scxml* which is created in the constructor of the state machine object.

```
model.cur_state = get_state<scxml>();
```

where get_state<> creates a singleton state object:

```
template<class T> T* get_state() {
    static T t;
    return &t;
}
```

scxml can be viewed as the *top* state. It is built-in and has an initial transition to the first application state, typically named *root*. Further nested states are then added under *root*.

Each concrete state can be automatically upcasted to the composite<> state above it for type matching (e.g. in *composite<>::enter()* and *exit()*).

An event handler (virtual) in a subclass overrides any handlers for the same event in all parent classes. This is analogous to a transition *sourced at* (originating from) a child state overrides any transitions triggered by the same event in all parent states. If an event is not handled, the base *state* provides a default handler returning 0 (to prevent updating the *cur_state* in *dispatch_event()* and break the initial transition loop in *dispatch_int()*).

```
struct state_active : public composite<state_active, scxml>
{
    state* initial(sc_vending_machine &sc) {
        return transition<&state::initial, state_active, state_collect_coins, internal>()(this, sc); }
    state* event_cancel(sc_vending_machine &sc) { state* s;
        return (s = transition<&state::event_cancel, state_active, state_coin_return>()(this, sc))
            || (s = parent_t::event_cancel(sc)), s; }
    state* event_done(sc_vending_machine &sc) { state* s;
        return (s = transition<&state::event_done, state_active, state_coin_return>()(this, sc))
            || (s = parent_t::event_done(sc)), s; }
    state* event_test(sc_vending_machine &sc) { state* s;
        return (s = transition<&state::event_test, state_active, state_collect_coins, internal>()(this, sc))
            || (s = transition<&state::event_test, state_active, state_dispense_coke, internal>()(this, sc))
            || (s = parent_t::event_test(sc)), s; }
};
```

Multiple transitions triggered by the same event and *sourced at* the same state but with different destination are *OR'ed* (||) together. The order of evaluation follows the order of appearances in the SCXML file. In practice, these transitions should be guarded by disjoint (non-overlapping) conditions such that at most one of them is enabled (guard evaluated to true). If all of them are disabled, the last *OR'ed* clause propagates the event to the parent state, invoking the closest (most derived) overriding event handler at or above the parent. (From now on, an event handler is selected by inheritance rather than virtual function.) It iterates until an enabled transition is found (i.e. transition *call operator* returning a non-zero pointer to the next/destination state) or the base state is reached (which by default returns 0).

An initial transition is handled like an event named *initial* (called the *initial event*). However, when an initial transition is disabled (guard evaluated to false), the *initial event* is not propagated to the parent state and its handler simply returns 0 to break the call chain in *dispatch_int()*. It is *very important* to note that the composite<> class (intermediate base class above each concrete state) implements a default initial handler that returns 0:

```
    template<class C, class P> class composite : public P, public state_actions<C>
    {
        virtual state* initial(sc_vending_machine&) { return 0; }
```

It provides a default initial handler for each concrete state such that it will be called (via virtual function selection) if the concrete state does not implement one (i.e. reaching a leaf state with no more initial transitions). Otherwise, the previous initial transition would be fired repeatedly (as the initial

handler of the parent state is called).

Here is a limitation. It does not support multiple transitions with the same (E, S, D) (i.e. a transition triggered by the same event with the same source and destination) even though they have different guard conditions. This is because they would have the same template parameters prohibiting multiple template instantiations. A work around is to raise different internal events via a targetless transition (triggered by the original event) within the state.

7 dispatch_int() and dispatch_event()

For portability, we would implement our own external and internal event queues out of scxmlcc and use it only as an HSM processor. The following describes the original scxmlcc with built-in queues.

An event (in form of a pointer to a member event handler) is dispatched to the state machine via the base class pointer (*state* *) named *cur_state* pointing to the current leaf state. The virtual event handler is called on the *cur_state* object, invoking the overriding handler of the most derived class corresponding to the most nested state. It matches the statechart rule that a child state takes precedence over parent states in selecting a transition (resolving conflicts). If the event handler returns a non-zero value, it represents the next state and is used to update *cur_state* (which can be unchanged). It returns 0 when the event is unhandled (default handling by the base *state*).

```
    private: bool dispatch_event(event e)
    {
        state *next_state;
        if ((next_state = (model.cur_state->*e)(*this))) model.cur_state = next_state;
        return !!next_state;
    }
```

The higher level dispatch function (*dispatch_int()*) handles the initial transitions from (but not including) the transition target to the final leaf state (see **composite** section, stage 3). It is done by dispatching the *initial event* in a loop as long as it is unhandled. As the loop iterates, the current state *cur_state* is updated to go to a more nested state. When the current state no longer defines an initial transition (i.e. no longer overrides the initial event handler), the default initial handler in the composite<> state above it will be called which returns 0 and breaks the loop.

```
public: void dispatch_int(event e = &state::unconditional)
{
    bool cont = dispatch_event(e) || dispatch_event(&state::unconditional);
    while (cont) {
        if ((cont = dispatch_event(&state::initial)));
        else if ((cont = dispatch_event(&state::unconditional)));
        else if (model.event_queue_int.size()) cont = dispatch_event(model.event_queue_int.front()),
            model.event_queue_int.pop_front(), cont |= !model.event_queue_int.empty();
        else break;
    }
}
```